

스키마와 참조구현, 어떻게 결정하나

작성자: 김정태 (hankkt@gmail.com)

작성일시: 2026. 8. 18.

근거: 3_근거_국제선례 — 국제 표준 16종의 배포 실물과 표준화 기구 6곳의 규정을 조사한 결과

무엇을 정해야 하나

표준을 제안하면 문서 말고도 두 가지가 팔려 나온다.

	무엇인가	예
스키마	기계가 읽을 수 있게 요소를 정의한 파일	이 표준의 "수온" 항목은 이런 뜻이고, 숫자여야 하고, 단위는 섭씨다 — 를 기계가 읽는 형식으로 쓴 것
참조구현 (선택)	그 스키마를 실제로 써먹는 예제 프로그램	데이터가 스키마에 맞는지 검사해주는 파이썬 코드

둘은 성격이 다르고, 그래서 표준에서의 지위도 이미 갈려 있다.

	표준의 일부인가	근거
스키마	그렇다. 표준의 규범 내용이다	운영요령 제2조제1항제8호
참조구현	아니다. 표준의 일부가 아니다	제2조제1항제9호

지위는 판단할 것이 아니라 무엇인지에서 따라 나온다. 정해야 할 것은 그다음이다.

정할 것	스키마	참조구현
표준 문서 안에 넣나	넣는다. 어떻게 넣을지가 문제 (1-1)	넣지 않는다. 부록에 위치만 (2-1)
남이 가져다 써도 되나	정해야 한다 (1-2)	정해야 한다 (2-2)

라이선스는 둘 다 정해야 하고, 서로 달라도 된다.

1부. 스키마

1-1. 문서 안에 넣을까, 밖에 둘까

왜 고민이 되나

둘 다 문제가 있어 보이기 때문이다.

	좋은 점	나쁜 점
안에 넣으면	표준의 일부가 된다. "이걸 지켜야 한다"고 말할 수 있다	기계가 못 읽는다. 한글 문서나 PDF에서 스키마를 꺼내야 한다
밖에 두면	기계가 주소만 알면 바로 가져간다	표준의 일부가 아닌 것처럼 보인다

기계가 읽으라고 만든 파일을, 기계가 못 읽는 곳에 넣는 셈이다. 그렇다고 빼면 "이건 그냥 참고 자료 아닌가"가 된다.

국제 사례를 보니 답이 있었다

표준 16종의 배포본을 실제로 내려받아 확인했다.

표준 문서 파일 안에 스키마를 담은 사례 — 0건

한 건도 없다. 모두 문서 밖에 별도 파일로 제공한다. 그중 가장 흔한 방식은 문서와 같은 자리에 파일을 나란히 두는 것이다.

미국 의회도서관의 보존 메타데이터 표준 (PREMIS)이 그 예다.

/standards/premis/v3/premis-3-0-final.pdf ← 표준 문서
/standards/premis/v3/premis.xsd ← 스키마 파일

같은 폴더에 나란히 있다. 문서를 찾으면 스키마도 눈에 들어오고, 기계는 스키마 주소만 알면 문서를 거치지 않고 바로 가져간다.

DDI(사회과학 데이터), OGC(지리정보), ISO(지리정보)도 같은 방식이다.

그래서 답은 "둘 다 한다"

둘 중 하나를 고르는 문제가 아니었다.

부속서에 수록하고, 같은 내용을 파일로도 낸다.

이것을 병행형이라 부른다.

- 부속서에 넣으니 → 표준의 일부가 된다 (작성지침이 부속서에 "본 부속서는 표준 내용의 일부임"을 적게 한다)
- 파일로도 내니 → 기계가 바로 가져간다

두 개가 다르면 부속서에 실린 것이 정본이다.

예외 — 셋 중 하나면 밖에 둔다

산출물이 크거나 성격이 다르면 부속서에 못 넣는다. 이때는 부속서에 "어디에 무엇이 있다"는 명세만 쓰고 실물은 밖에 둔다. 이것을 참조형이라 한다.

파일이 밖에 있어도 규범이다. 부속서에 실린 명세가 "이 주소의 이 버전이 규범이다"라고 지정하기 때문이다.

사유	무슨 뜻인가	실제 사례
① 너무 크다	문서에 넣을 분량이 아니다	의료 표준 HL7 FHIR은 스키마 파일이 323개, 지리정보 ISO 표준은 117개다
② 자주 바뀐다	표준 문서보다 스키마가 더 자주 개정된다	부속서에 넣으면 스키마를 고칠 때마다 표준 개정 절차를 처음부터 밟아야 한다
③ 여러 파일로 쪼개져 있다	한 파일이 다른 파일들을 불러오는 구조다	OGC의 gml.xsd 는 그 자체로는 껍데기이고 다른 파일 28개를 불러온다. 그 파일만 받으면 작동하지 않는다

②가 실무에서 가장 자주 걸린다. 분량이 작아도 자주 바뀌면 부속서에 넣는 순간 개정 부담이 생긴다.

참조형의 대표 사례는 ECMA-424다. 소프트웨어 부품 목록(SBOM) 국제표준인데, 566쪽 본문에 스키마 코드가 한 줄도 없다. 대신 "이 주소의 이 버전이 규범이다"라고 가리킨다.

산출물마다 따로 정한다

한 표준이 여러 종류를 낼 수 있고, 각각 형태가 달라도 된다.

산출물	보통 어느 쪽	왜
요소↔어휘 매핑표 (@context)	병행형	표 하나 분량이고 문서와 함께 개정된다
검증용 스키마 (JSON Schema • XSD)	경우에 따라	파일 하나면 병행형, 여러 개로 쪼개져 있으면 참조형

1-2. 라이선스를 어떻게 정하나

왜 필요한가 — 안 적으면 못 쓴다

실제로 겪은 사례가 있다.

ISO의 지리정보 표준. 스키마 파일 117개를 무료로 공개한다. 그런데 파일 어디에도 이용조건이 없다. 사이트 약관에 걸려 있다고 하는데, 그 약관 페이지는 접속이 막혀 있었다.

받아갈 수는 있다. 그런데 복사해서 써도 되는지, 고쳐도 되는지, 우리 표준에 인용해도 되는지 알 수가 없다.

권리를 주장할 생각이 없어도, 적어두지 않으면 이용자는 쓰지 못한다.

반대 사례도 있다. 미국 의회도서관의 METS는 스키마 파일 안에 이렇게 적어 두었다.

"디지털도서관연합은 이 문서의 창작자로서, 저작권법상 이 문서에 대한 모든 권리를 전 세계적으로 포기하였다"

파일 하나만 받아도 조건이 확인된다.

어느 라이선스를 쓸까

CC BY 4.0(저작자표시)을 권한다.

	판단	이유
CC BY 4.0	권고	출처만 밝히면 자유롭게 쓸 수 있다. 표준 문서와 같은 계열이라 일관되고, 용어 목록 같은 데이터까지 함께 다룬다
CC BY-SA	피한다	"동일조건변경허락" — 이걸 가져다 만든 것도 같은 조건으로 공개해야 한다. 산업계가 확장 어휘를 만들기 어려워진다. Schema.org가 이 조건이라 정의문을 옮겨 적을 때 주의가 필요하다
Apache 2.0	차선	특히 조항이 있어 좋지만, 변경이력 고지 같은 소프트웨어식 의무가 표준 배포 관행과 잘 맞지 않는다
CC0 (권리 포기)	가능	최대한 널리 퍼뜨리는 것이 목표라면. 다만 한국법에서는 권리 포기가 제한되어 안전장치에 기대야 한다
아무것도 안 적기	금지	위 ISO 사례가 그 결과다

어디에 적나 — 파일 안에 적어야 한다

"부속서에 넣었으니 표준 문서의 저작권 문구가 적용되는 것 아닌가?"

그렇지 않다. 국제 선례가 이 점을 분명히 한다.

IETF(인터넷 표준화 기구)는 표준 문서 안에 코드를 넣는데, 그 코드를 <CODE BEGINS>라는 표시로 구분하고 문서와 다른 라이선스를 적용한다. 그리고 이렇게 못 박았다.

"문서에 정한 추가 허락과 제한은 그 코드에 적용되지 아니한다"

문서에 담겼다고 문서의 조건이 자동으로 따라가지 않는다는 뜻이다. 그래서 부속서에 수록하더라도 파일에 따로 적어야 한다.

형식마다 적는 방법이 다르다

형식	방법	실제로 이렇게 하는 곳
----	----	--------------

XSD · Turtle	파일 맨 위에 주석으로	CycloneDX
JSON Schema	주석을 쓸 수 없다. '\$comment' 항목에 적는다	CycloneDX
RDF 어휘	dcterms:license 한 줄로 — 기계가 읽을 수 있다	W3C DCAT

JSON 형식은 문법상 주석이 없다. 그래서 CycloneDX는 \$comment라는 항목을 우회 수단으로 쓴다.

특허는 라이선스로 풀지 않는다

CC 계열은 저작권만 다루고 특허는 건드리지 않는다. CC BY 4.0 본문에 "특허권과 상표권은 이 라이선스로 허락되지 아니한다"고 적혀 있다.

그렇다고 특허 조항이 있는 소프트웨어 라이선스로 바꿀 필요는 없다. TTA에는 이미 지식재산권 협약 제도가 있다. 스키마를 표준의 일부로 보게 해 두면 그 제도가 그대로 적용된다. 라이선스가 아니라 제도로 푸는 문제다.

작동 방식은 이렇다.

스키마 --[제19조의3제10항]--> 표준초안의 일부로 본다

운영규정 제36조	-----	표준초안에 지식재산권이 포함되면 처리 의무
운영요령 제19조① 1호	-----	초안 작성시 확인 + 협약서 제출 고지
운영요령 제23조③	-----	협약 거부·실시 허락 거절 시 표준 폐지

라이선스보다 강한 장치다. Apache 2.0의 방어적 종료는 "소송을 걸면 당신의 특허 실시권이 소멸한다"에 그치지만, TTA는 협약을 거부하면 표준 자체를 폐지한다.

이 제도는 참조구현에는 미치지 않는다. 제19조의3제10항의 대상은 기계가독 산출물뿐이다. 참조구현은 표준의 일부가 아니므로(제8항) 확인 대상에서 벗어난다. 실행 가능한 코드까지 넣으면 그 코드가 의존하는 라이브러리 연쇄까지 대상이 되어 통제할 수 없기 때문이다. → 2-2

2부. 참조구현

2-1. 문서에 넣을까 — 넣지 않는다

고민할 것이 없다. 부록에 "여기에 있습니다"만 적는다.

이유가 둘이다.

첫째, 코드는 금방 낡는다. 특정 언어와 버전으로 짠 예제를 표준에 박아 두면 몇 년 뒤에는 실행되지 않는다. 표준은 10년을 가는데 코드는 그렇지 않다.

둘째, 지식재산권 확인 범위가 건잡을 수 없이 넓어진다. 코드에 표준의 지위를 주면 그 코드가 쓰는 라이브러리, 그 라이브러리가 쓰는 또 다른 라이브러리까지 확인 대상이 될 수 있다.

그래서 부록에 위치만 안내하고, 협회는 그 내용을 보증하지 않는다고 밝힌다.

2-2. 라이선스는 스키마와 달라도 된다

같이 맞출 이유가 없다. Schema.org가 실제로 나눠 쓴다.

Schema.org의 선택	
어휘 (스키마 자체)	CC BY-SA 3.0 — 콘텐츠 라이선스
빌드 도구(코드)	Apache 2.0 — 소프트웨어 라이선스

W3C도 문서용 라이선스와 소프트웨어용 라이선스를 따로 둔다.

참조구현은 실제로 실행되는 프로그램이라 특허 문제가 실제로 생긴다. CC 계열은 특허를 다루지 않으므로 맞지 않고, 특허 조항이 있는 소프트웨어 라이선스(Apache 2.0 등)가 적합하다.

다만 표준 밖의 자산이므로 제안자가 정하면 된다. 규정은 "어디에 있고 조건이 무엇인지 밝혀라"까지만 요구한다.

그래서 라이선스 선택이 스키마보다 중요하다. 스키마는 TTA의 지식재산권 확약 제도가 특허를 받쳐 주지만, 참조구현은 그 제도 밖이라 라이선스가 유일한 장치다. 특허 조항이 없는 MIT·BSD로 내면 이용자가 특허 위험을 그대로 진다.

이용자가 판단할 수 있도록 제안 단계 선언서(별지 제11-1호) 제6항에서 참조구현의 위치·언어·런타임·이용조건을 밝히게 한다.

3부. 결정 순서

① 이것이 무엇인가 — 성격으로 가른다

데이터의 구조·의미를 정의하는가?

예: 요소 정의를 담은 JSON Schema · XSD · @context · RDF 어휘
→ 스키마. 표준의 규범 내용이다. ②로

그 정의를 사용·처리하는 프로그램인가?

예: 데이터가 스키마에 맞는지 검사하는 코드, 변환 스크립트
→ 참조구현. 표준의 일부가 아니다.

부록에 위치만 안내하고, ③에서 라이선스만 정한다

※ 규범 여부는 여기서 판단하는 것이 아니라 성격에서 따라 나온다.
운영요령 제2조제1항제8호가 기계가독 산출물을 "표준의 규범 내용"으로,
제9호가 참조구현을 "표준의 일부가 아니"라고 이미 정하고 있다.

※ 경계가 애매하면 — 변환기·검증기처럼 정의와 처리에 걸친 산출물은
제안 단계 선언(별지 제11-1호)에서 어느 쪽으로 낼지 밝힌다.

② 스키마를 부속서에 넣을 수 있는가? 아래 셋을 모두 통과해야 한다

- 분량이 문서에 들어가는가?
- 표준 문서와 같은 주기로 개정되는가?
- 파일 하나로 완결되는가? (다른 파일을 불러오지 않는가?)

셋 다 예 —————→ 병행형 : 부속서에 전문 수록 + 파일 제공

하나라도 아니오 -----→ 참조형 : 부속서에 명세만 수록, 실물은 밖에
 ※ 실물이 밖에 있어도 규범이다

③ 라이선스를 정한다 (①②의 결과와 무관하게 둘 다 반드시 필요)

스키마 → CC BY 4.0 권고
 · 동일조건변경허락(SA) 조건은 피한다
 · 아무것도 안 적는 것은 금지
 참조구현 → 제안자가 정하되, 특허 조항이 있는 코드 라이선스를 권한다

④ 형식에 맞는 방법으로 파일에 적는다 (스키마는 부속서에 넣었더라도)

XSD · Turtle → 파일 머리말 주석
 JSON Schema → \$comment 항목
 RDF 어휘 → dct:license 한 줄

4부. 한 장 요약

	문서에 넣나	라이선스	어디에 적나
스키마	병행형이 원칙 / (부속서 수록 + 파일 제공) / / 크거나 · 자주 바뀌거나 · 여러 파일이면 참조형	CC BY 4.0 권고 / / SA 조건 피함 / 무표기 금지	파일 자체에 필수 / 부속서에 넣었어도 별도로
참조구현	항상 밖 / 부록에 위치만 안내	제안자 자율 / 특허 조항 있는 코드 라이선스 권고	코드 저장소

부록. 이 판단의 근거

각 판단이 어느 조사에서 나왔는지 밝힌다.

판단	근거	어느 조사
부속서에만 넣지 않는다	표준 문서에 스키마를 담은 사례 0건	3-1 (16종 실물)
병행형이 원칙이다	LoC PREMIS · MODS · METS, DDI, OGC, ISO의 병행 게시	3-1
참조형 예외 — 분량	HL7 FHIR 323개, ISO 117개	3-1
참조형 예외 — 모듈 의존	OGC gml.xsd 가 다른 파일 28개를 불러옴	3-1
참조형의 실제 모습	ECMA-424 — 566쪽에 스키마 코드 0줄	3-1
무표기의 폐해	ISO 19115-3 — XSD 117개에 이용 조건 없음, 약관 접근 불가	3-1
파일 내 표기의 모범	LoC METS(권리 포기 선언), DDI 3.3(CC BY 4.0)	3-1
형식별 표기 방법	CycloneDX \$comment , W3C	3-1

	dcterms:license 트리플	
문서에 담겨도 조건이 자동 적용 되지 않음	IETF 규정 제4조 — 코드에 다른 라이선스, 문서 제한 배제	3-2
문서용 · 소프트웨어용 라이선스 분리	W3C의 라이선스 이원화, Schema.org의 어휘/도구 분리	3-2 + 3-1
특허를 제도로 푸는 방식	W3C 특허정책 — 라이선스와 별도 로 기구 정책이 담당	3-2

3-1은 "실제로 이렇게 한다"를, 3-2는 "그 근거 규정이 이렇게"를 낸다. 두 조사의 관계는 3_근거_국제선례/README.md에 정리해 두었다.

관련 조문

이 문서의 판단이 규범 개정안의 어느 조문에 반영되었는지.

판단	조문
병행형 원칙	운영요령 제19조의3제2항
참조형 예외 3사유	제19조의3제3항
참조 명세 기재사항	제19조의3제4항
협회 보관본이 정본	제19조의3제6항
파일 자체에 이용조건 표시	제19조의3제7항
참조구현은 표준 밖	제19조의3제8항
지식재산권 확인 대상 포함	제19조의3제10항
형식별 표기 방법	작성지침 6.6